Project Architecture

-GPTinho-

Problem and Data

Professional football coaching staff and data analysts spend countless hours manually cross-referencing massive, raw **event datasets** with qualitative, unstructured **scouting reports** to prepare match briefings. GPTinho addresses this gap by acting as a natural language **tactical assistant** constrained entirely to verified match datasets and internal club reports, synthesizing quantitative metrics and qualitative notes into cohesive answers.

To ensure the system can perform robust opponent form analysis without encountering data gaps, the prototype will be anchored to the complete 2015/2016 league season using the open-source **StatsBomb dataset**. Upon analysing this dataset we confirmed that it contains all 380 matches for the English, Spanish and Italian league seasons, guaranteeing **100% coverage** for all 20 teams per league. This structural completeness ensures the agent can accurately analyze true **opponent form** and historical **tactical trends**, which will be paired with a representative set of unstructured PDF scouting reports to form the system's complete knowledge base.

The system will support ingesting new data through two parallel automated pipelines. Raw, highly nested StatsBomb JSON event files will be processed, flattened, and loaded into a local **relational SQL database**, organized into structured tables for matches, players, and specific tactical events. Concurrently, unstructured PDF scouting reports will be processed via text extraction, chunked, embedded, and stored in a local **vector database** for semantic retrieval.

Golden Dataset

This dataset will serve as the absolute **ground truth** for testing the agent's retrieval and synthesis capabilities, completely separate from the model's training data. We will manually craft a representative set of 50 to 100 **tactical questions** based directly on the anchored 2015/2016 league fixtures and our corresponding scouting reports.

The questions will be stratified across three tactical tiers:

- purely quantitative queries that test the Text-to-SQL pipeline (e.g., player turnover rates)
- purely qualitative queries that test the vector search (e.g., pressing triggers noted by scouts)
- hybrid queries that require the orchestration of both data streams.

To enable strict **step-by-step verification**, a single entry in this dataset will contain the natural language prompt, the manually verified SQL query, the exact statistical rows that the query must return, the specific expected paragraph from the PDF scouting report, and a human-written final synthesized answer.

By comparing the agent's intermediate outputs against the Golden Dataset, we can mathematically verify the **Execution Accuracy** (EX) of the generated SQL against the StatsBomb schema. Furthermore, we will use evaluation frameworks like **RAGAS** to measure the context precision of the vector retrieval, ensuring the correct scouting notes are fetched

RAG Design

We will implement **Recursive Character Text Splitting** with a large window (1000 characters) and a significant overlap. This method prioritizes natural paragraph breaks, ensuring that a complete thought remains entirely intact within a single chunk. These chunks will then be processed using a local, open-weight embedding model: **nomic-embed-text**.

The embedded chunks will be stored in **ChromaDB**. This vector database is ideal for our prototype because it runs entirely locally and it integrates seamlessly with our Python backend. Finally, to retrieve this data accurately, we will rely heavily on **Metadata Pre-filtering**. A standard semantic search across all scouting reports risks pulling tactical notes about the wrong team simply due to similar vocabulary. To prevent this, our ingestion pipeline will tag every chunk with **team** and **match_date** metadata. When the agent queries the vector database, the MCP server will apply a strict metadata filter (e.g., **{"team": "Liverpool"}**), guaranteeing that the semantic search only retrieves qualitative insights specifically related to the targeted opponent.

Agents and Tools

The core of the system is a fine-tuned Small Language Model that operates as an autonomous **reasoning agent** rather than a simple chatbot. When a coach submits a tactical question, the agent must make complex routing and decisions. It first determines whether the query requires hard statistical data, qualitative insights, or a hybrid of both. It then translates the coach's natural language into the appropriate **technical parameters**, formulating either precise **SQL statements** for the statistical database or **semantic search strings** for the vector database. Once the requested data is retrieved, the agent performs a final synthesis step, merging raw statistical rows and unstructured PDF paragraphs into a cohesive tactical briefing.

To interact with the underlying data, the agent is equipped with two specific tools. The first is a relational database **querying tool** that executes the agent's generated SQL against the StatsBomb dataset to fetch quantitative **match metrics**. Crucially, this pipeline features an internal **error-correction loop**: if the generated SQL results in a syntax error or fails to execute, the error message is fed directly back to the agent. This allows the model to autonomously debug its own logic and retry the query before responding to the user. The second tool is a **vector search function** designed for the RAG pipeline. The agent passes a semantic search phrase along with a strict team-name metadata filter, and the tool retrieves the most relevant tactical paragraphs from the local ChromaDB.

The bridge connecting the agent's reasoning engine to these physical tools is the **Model Context Protocol** (MCP) server. The MCP server operates as a secure middleware layer, meaning the SLM never touches the local databases directly. Before the initial prompt even reaches the agent, the MCP server dynamically injects the relational database schema and the simulated matchday context (e.g the exact simulated date and the upcoming opponent). When the agent decides to invoke a tool, it sends a **structured request** to the MCP server, which validates the request, executes the tool securely, and returns the raw data to the agent for its final synthesis.

Example

- *Query*: "How does Liverpool build up when pressed high, and which midfielder has the most turnovers under pressure?"

- *Expected Retrieval*:
    - Quantitative Data (SQL): SELECT player.name, COUNT(event.id) as turnovers... expected to return exactly [{"name": "Emre Can", "turnovers": 18}].
    - Qualitative (RAG): Expected to retrieve Chunk #402: "When pressed high, Liverpool center-backs bypass the midfield pivot with direct, long passes to the wings."

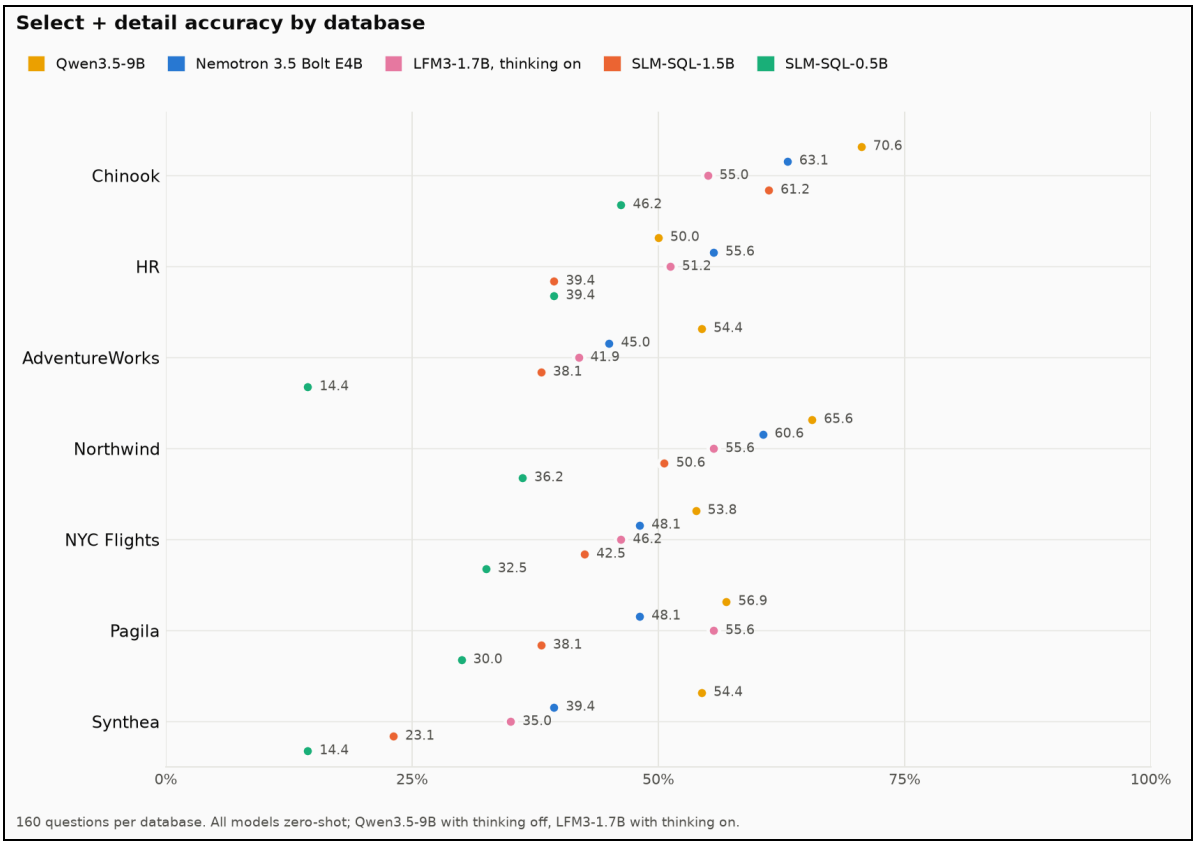
- *Agent Synthesis*: "Under a high press, Liverpool bypasses their midfield with long passes to the wings. Emre Can is their most vulnerable midfielder, with 18 turnovers under pressure."
- *Verification:*
    - Execution Accuracy (EX): The automated grader runs the agent's generated SQL against the StatsBomb database. If it returns exactly Emre Can, 18, the SQL pipeline passes.
    - Context Precision (RAGAS): The grader compares the vector search results to Chunk #402. If the agent retrieved the wrong team's notes, it fails.

Model Choices

The chosen model must possess **native function-calling** capabilities to interface with the MCP server and demonstrate **high-level reasoning** to navigate the complex, nested StatsBomb database schema.

Instead of relying on standard Exact Match (EM) text benchmarks, our selection criteria prioritize **Execution Accuracy** (EX), which verifies that the SQL generated by the model successfully executes against a relational database and returns the correct rows. Based on the Small LLM Text-to-SQL Benchmark, we have selected **Qwen3.5-9B** as our primary agent model. Testing on PostgreSQL databases reveals that Qwen3.5-9B currently leads the sub-10B parameter class, successfully answering 57.9% of complex data lookups. Its strict adherence to structured output makes it highly reliable for translating natural language into valid SQL within our MCP architecture.

As a secondary candidate for our experimental phase, we will also evaluate **Nemotron 3.5 Bolt E4B**. While Qwen3.5-9B excels at data retrieval, Nemotron demonstrates superior **safety guardrails**, correctly identifying and refusing 86.8% of unanswerable queries compared to Qwen's 72.7%. This capability to explicitly output a marker when data is missing is a critical feature for preventing statistical hallucinations. Both models will be tested against our custom Golden Dataset to determine which provides the optimal balance of SQL accuracy and operational safety for the final prototype.



Project Timeline

Weeks 7–8: Data & Infrastructure

- Ingest 2015/2016 season JSON data into local SQL tables and embed PDF scouting reports into ChromaDB.
- Begin drafting the Golden Dataset (focusing on expected SQL queries).

Weeks 9–10: MCP Server & Agent Setup

- Build the MCP middleware to handle context injection and secure tool execution.
- Deploy the local SLMs (Qwen3.5-9B and Nemotron) and code the Text-to-SQL and Vector Search functions.

Weeks 11–12: Evaluation & Checkup

- Finalize the Golden Dataset and write the automated evaluation script (using EX and RAGAS metrics).

Weeks 13–14: UI, Final Testing & Presentation

- Build the "Simulated Matchday" UI to dynamically anchor the timeline and prevent data leakage.
- Run the Golden Dataset against both models to finalize the SLM choice.